Esha

Your Code

```python
students = [
    ("Arun", [78, 85, 92, 67]),
    ("Bala", [45, 67, 55, 49]),
    ("Charan", [90, 92, 88, 95]),
    ("Divya", [65, 72, 70, 68]),
    ("Esha", [88, 76, 91, 84])
]

totals = []
averages = []

print("Student Totals:")
for name, marks in students:
    total = sum(marks)
    average = total / len(marks)
    totals.append(total)
    averages.append(average)
    print(name + ":", total)

print("\nStudent Averages:")
for i in range(len(students)):
    print(students[i][0] + ":", format(averages[i], ".2f"))


highest = max(totals)
print("\nHighest Total:", students[totals.index(highest)][0])


lowest = min(averages)
print("Lowest Average:", students[averages.index(lowest)][0])


class_average = sum(averages) / len(averages)
print("\nAbove Class Average:")
for i in range(len(students)):
    if averages[i] > class_average:
        print(students[i][0], end=" ")


print("\n\nStudents with 3 or more marks above 80:")
for name, marks in students:
    if sum(mark > 80 for mark in marks) >= 3:
        print(name)


print("\nSubject-wise Highest:")
for i in range(4):
    highest_mark = max(marks[i] for name, marks in students)
    print("Subject", i + 1, ":", highest_mark)


print("\nStudents with a mark below 50:")
for name, marks in students:
    if any(mark < 50 for mark in marks):
        print(name)


sorted_students = sorted(
    zip(students, averages),
    key=lambda x: x[1],
    reverse=True
)

print("\nSorted by Average:")
for (name, marks), average in sorted_students:
    print(name, ":", format(average, ".2f"))


print("\nTop 2:")
for i in range(2):
    print(sorted_students[i][0][0])


print("\n10. Structure:")
print("Tuple is immutable.")
print("List inside the tuple is mutable.")
```

Input

```
print("Student Totals:")
for name, marks in students:
    total = sum(marks)
    average = total / len(marks)
    totals.append(total)
    averages.append(average)
    print(name + ":", total)

print("\nStudent Averages:")
for i in range(len(students)):
    print(students[i][0] + ":", format(averages[i], ".2f"))


highest = max(totals)
print("\nHighest Total:", students[totals.index(highest)][0])


lowest = min(averages)
print("Lowest Average:", students[averages.index(lowest)][0])


class_average = sum(averages) / len(averages)
print("\nAbove Class Average:")
for i in range(len(students)):
    if averages[i] > class_average:
        print(students[i][0], end=" ")


print("\n\nStudents with 3 or more marks above 80:")
for name, marks in students:
    if sum(mark > 80 for mark in marks) >= 3:
        print(name)


print("\nSubject-wise Highest:")
for i in range(4):
    highest_mark = max(marks[i] for name, marks in students)
    print("Subject", i + 1, ":", highest_mark)


print("\nStudents with a mark below 50:")
for name, marks in students:
    if any(mark < 50 for mark in marks):
        print(name)


sorted_students = sorted(
    zip(students, averages),
    key=lambda x: x[1],
    reverse=True
)

print("\nSorted by Average:")
for (name, marks), average in sorted_students:
    print(name, ":", format(average, ".2f"))


print("\nTop 2:")
for i in range(2):
    print(sorted_students[i][0][0])


print("\n10. Structure:")
print("Tuple is immutable.")
print("List inside the tuple is mutable.")
```

Input

```
stdin input
```

Output

```
Student Totals:
Arun: 322
Bala: 216
Charan: 365
Divya: 275
Esha: 339
```

Run Save

Your Code

```python
names = [
    "Arun Kumar ",
    "ARUN KUMAR",
    " arun kumar",
    "Bala Krishnan",
    "BALA KRISHNAN ",
    "Charan",
    " charan ",
    "DIVYA DEVI",
    "Divya Devi"
]


normalized = [name.strip().lower() for name in names]


duplicates = []
for name in normalized:
    if normalized.count(name) > 1 and name not in duplicates:
        duplicates.append(name)


unique = list(dict.fromkeys(normalized))


multiple_words = [name for name in unique if len(name.split()) > 1]


longest = max(unique, key=len)


a_or_d = [name for name in unique if name.startswith('a') or name.start


character_count = {name: len(name) for name in unique}


sorted_names = sorted(unique)


most_frequent = max(unique, key=normalized.count)

print("Normalized:")
print(normalized)

print("\nUnique Customers:")
print(unique)

print("Number of Unique Customers:", len(unique))

print("\nDuplicate Customers:")
for name in duplicates:
    print(name)

print("\nNames with Multiple Words:")
for name in multiple_words:
    print(name)

print("\nLongest Customer Name:")
print(longest)

print("\nCustomers starting with 'a' or 'd':")
for name in a_or_d:
    print(name)

print("\nCharacter Count:")
for name, count in character_count.items():
    print(name, ":", count)

print("\nSorted Unique Names:")
print(sorted_names)

print("\nMost Frequently Occurring Customer:")
print(most_frequent)
```

Input

```
stdin input
```

```
    "DIVYA DEVI",
    "Divya Devi"
]


normalized = [name.strip().lower() for name in names]


duplicates = []
for name in normalized:
    if normalized.count(name) > 1 and name not in duplicates:
        duplicates.append(name)


unique = list(dict.fromkeys(normalized))


multiple_words = [name for name in unique if len(name.split()) > 1]


longest = max(unique, key=len)


a_or_d = [name for name in unique if name.startswith('a') or name.start


character_count = {name: len(name) for name in unique}


sorted_names = sorted(unique)


most_frequent = max(unique, key=normalized.count)

print("Normalized:")
print(normalized)

print("\nUnique Customers:")
print(unique)

print("Number of Unique Customers:", len(unique))

print("\nDuplicate Customers:")
for name in duplicates:
    print(name)

print("\nNames with Multiple Words:")
for name in multiple_words:
    print(name)

print("\nLongest Customer Name:")
print(longest)

print("\nCustomers starting with 'a' or 'd':")
for name in a_or_d:
    print(name)

print("\nCharacter Count:")
for name, count in character_count.items():
    print(name, ":", count)

print("\nSorted Unique Names:")
print(sorted_names)

print("\nMost Frequently Occurring Customer:")
print(most_frequent)
```

Input

```
stdin input
```

Output

```
Normalized:
['arun kumar', 'arun kumar', 'arun kumar', 'bala krishnan', 'bala krishnan',
'charan', 'charan', 'divya devi', 'divya devi']

Unique Customers:
['arun kumar', 'bala krishnan', 'charan', 'divya devi']
```

Run Save

Laptop

Highest Inventory Value:
Laptop

Total Quantity: 69

Your Code

```python
products = [
    ("Laptop", 5, 55000),
    ("Mouse", 25, 700),
    ("Keyboard", 8, 1200),
    ("Monitor", 12, 15000),
    ("Printer", 4, 12000),
    ("Headset", 15, 2500)
]


print("Inventory Value:")
for name, quantity, price in products:
    print(name, ":", quantity * price)


total_value = sum(quantity * price for name, quantity, price in product
print("\nTotal Inventory Value:", total_value)


print("\nProducts with Quantity Less Than 10:")
for name, quantity, price in products:
    if quantity < 10:
        print(name)


print("\nProducts with Price Greater Than 10000:")
for name, quantity, price in products:
    if price > 10000:
        print(name)


expensive = max(products, key=lambda x: x[2])
print("\nMost Expensive Product:", expensive[0])


highest_value = max(products, key=lambda x: x[1] * x[2])
print("Highest Inventory Value Product:", highest_value[0])


print("\nProducts Sorted by Price:")
for product in sorted(products, key=lambda x: x[2]):
    print(product)


print("\nProducts Sorted by Quantity:")
for product in sorted(products, key=lambda x: x[1]):
    print(product)


total_quantity = sum(quantity for name, quantity, price in products)
print("\nTotal Quantity:", total_quantity)


print("\nProducts with Inventory Value More Than 350000:")
for name, quantity, price in products:
    if quantity * price > 350000:
        print(name)
```

Input

```
stdin input
```

Output

```
Inventory Value:
Laptop : 275000
Mouse : 17500
Keyboard : 9600
Monitor : 180000
Printer : 48000
```

Run Save

Analysis – String + List + Tuple
10 marks

10/10 answered

Words containing 'p':
python
programming
powerful
practice

Your Code

```python
feedback = """Python programming is easy to learn.
Python programming is powerful.
Learning Python requires practice.
Practice makes programming skills better."""


text = feedback.lower()


for ch in ".,!?":
    text = text.replace(ch, "")


words = text.split()


total_words = len(words)


unique_words = set(words)


python_count = words.count("python")


programming_count = words.count("programming")


repeated = []
for word in unique_words:
    if words.count(word) > 1:
        repeated.append(word)


longest = max(words, key=len)


sorted_words = sorted(unique_words)


words_with_p = [word for word in unique_words if 'p' in word]


percentage = (len(unique_words) / total_words) * 100

print("Lowercase Text:")
print(text)

print("\nWords:")
print(words)

print("\nTotal Number of Words:", total_words)

print("Number of Unique Words:", len(unique_words))

print("Frequency of 'python':", python_count)

print("Frequency of 'programming':", programming_count)

print("\nWords Appearing More Than Once:")
for word in sorted(repeated):
    print(word)

print("\nLongest Word:", longest)

print("\nUnique Words in Alphabetical Order:")
print(sorted_words)

print("\nWords Containing 'p':")
print(sorted(words_with_p))

print("\nPercentage of Unique Words:", round(percentage, 2), "%")
```

```python
for ch in '.,!?':
    text = text.replace(ch, "")


words = text.split()


total_words = len(words)


unique_words = set(words)


python_count = words.count("python")


programming_count = words.count("programming")


repeated = []
for word in unique_words:
    if words.count(word) > 1:
        repeated.append(word)


longest = max(words, key=len)


sorted_words = sorted(unique_words)


words_with_p = [word for word in unique_words if 'p' in word]


percentage = (len(unique_words) / total_words) * 100

print("Lowercase Text:")
print(text)

print("\nWords:")
print(words)

print("\nTotal Number of Words:", total_words)

print("Number of Unique Words:", len(unique_words))

print("Frequency of 'python':", python_count)

print("Frequency of 'programming':", programming_count)

print("\nWords Appearing More Than Once:")
for word in sorted(repeated):
    print(word)

print("\nLongest Word:", longest)

print("\nUnique Words in Alphabetical Order:")
print(sorted_words)

print("\nWords Containing 'p':")
print(sorted(words_with_p))

print("\nPercentage of Unique Words:", round(percentage, 2), "%")
```

Input

```
stdin input
```

Output

```
Lowercase Text:
python programming is easy to learn
python programming is powerful
learning python requires practice
practice makes programming skills better
```

Run Save

Your Code

```python
employees = [
    ("E101", "Arun", ["Python", "SQL", "ML"]),
    ("E102", "Bala", ["Java", "SQL", "HTML"]),
    ("E103", "Charan", ["Python", "ML", "SQL"]),
    ("E104", "Divya", ["Python", "Java", "Cloud"]),
    ("E105", "Esha", ["Python", "ML", "Cloud"])
]


print("Employees who know Python:")
for emp in employees:
    if "Python" in emp[2]:
        print(emp[1])


print("\nEmployees who know Python and ML:")
for emp in employees:
    if "Python" in emp[2] and "ML" in emp[2]:
        print(emp[1])


print("\nEmployees who know SQL but not Java:")
for emp in employees:
    if "SQL" in emp[2] and "Java" not in emp[2]:
        print(emp[1])


print("\nEmployees who know Java or Cloud:")
for emp in employees:
    if "Java" in emp[2] or "Cloud" in emp[2]:
        print(emp[1])


skill_count = {}

for emp in employees:
    for skill in emp[2]:
        if skill in skill_count:
            skill_count[skill] += 1
        else:
            skill_count[skill] = 1

print("\nSkill Count:")
print(skill_count)


most_common = max(skill_count, key=skill_count.get)
print("\nMost Common Skill:", most_common)


print("\nEmployees having at least 3 skills:")
for emp in employees:
    if len(emp[2]) >= 3:
        print(emp[1])


print("\nEmployees who do not know Python:")
for emp in employees:
    if "Python" not in emp[2]:
        print(emp[1])


print("\nEmployees sorted by number of skills:")
sorted_employees = sorted(employees, key=lambda x: len(x[2]))

for emp in sorted_employees:
    print(emp[1], ":", len(emp[2]), "skills")


print("\nEmployees suitable for Python + ML project:")
for emp in employees:
    if "Python" in emp[2] and "ML" in emp[2]:
        print(emp[1])
```

Input

stdin input

```python

print("Employees who know Python:")
for emp in employees:
    if "Python" in emp[2]:
        print(emp[1])


print("\nEmployees who know Python and ML:")
for emp in employees:
    if "Python" in emp[2] and "ML" in emp[2]:
        print(emp[1])


print("\nEmployees who know SQL but not Java:")
for emp in employees:
    if "SQL" in emp[2] and "Java" not in emp[2]:
        print(emp[1])


print("\nEmployees who know Java or Cloud:")
for emp in employees:
    if "Java" in emp[2] or "Cloud" in emp[2]:
        print(emp[1])


skill_count = {}

for emp in employees:
    for skill in emp[2]:
        if skill in skill_count:
            skill_count[skill] += 1
        else:
            skill_count[skill] = 1

print("\nSkill Count:")
print(skill_count)


most_common = max(skill_count, key=skill_count.get)
print("\nMost Common Skill:", most_common)


print("\nEmployees having at least 3 skills:")
for emp in employees:
    if len(emp[2]) >= 3:
        print(emp[1])


print("\nEmployees who do not know Python:")
for emp in employees:
    if "Python" not in emp[2]:
        print(emp[1])


print("\nEmployees sorted by number of skills:")
sorted_employees = sorted(employees, key=lambda x: len(x[2]))

for emp in sorted_employees:
    print(emp[1], ":", len(emp[2]), "skills")


print("\nEmployees suitable for Python + ML project:")
for emp in employees:
    if "Python" in emp[2] and "ML" in emp[2]:
        print(emp[1])
```

Input

```
stdin input
```

Output

```
Employees who know Python:
Arun
Charan
Divya
Esha
```

Run Save

Your Code

```python
transactions = [
    ('T101', 'Deposit', 5000),
    ('T102', 'Withdraw', 2000),
    ('T103', 'Deposit', 8000),
    ('T104', 'Withdraw', 1500),
    ('T105', 'Deposit', 3000),
    ('T106', 'Withdraw', 7000)
]


total_deposit = sum(amount for id, type, amount in transactions if type


total_withdraw = sum(amount for id, type, amount in transactions if typ


net_balance = total_deposit - total_withdraw


largest_deposit = max(amount for id, type, amount in transactions if ty


largest_withdraw = max(amount for id, type, amount in transactions if t


greater_4000 = [t for t in transactions if t[2] > 4000]


deposit_count = sum(1 for id, type, amount in transactions if type == "
withdraw_count = sum(1 for id, type, amount in transactions if type ==


average = sum(t[2] for t in transactions) / len(transactions)


sorted_transactions = sorted(transactions, key=lambda x: x[2])


result = total_withdraw > (total_deposit * 0.50)

print("Total Deposits:", total_deposit)
print("Total Withdrawals:", total_withdraw)
print("Net Balance:", net_balance)
print("Largest Deposit:", largest_deposit)
print("Largest Withdrawal:", largest_withdraw)

print("\nTransactions greater than 4000:")
for t in greater_4000:
    print(t)

print("\nNumber of Deposits:", deposit_count)
print("Number of Withdrawals:", withdraw_count)

print("Average Transaction Amount:", average)

print("\nTransactions Sorted by Amount:")
for t in sorted_transactions:
    print(t)

print("\nWithdrawals exceed 50% of deposits:", result)
```

Input

```
stdin input
```

Output

```
Total Deposits: 16000
Total Withdrawals: 10500
Net Balance: 5500
Largest Deposit: 8000
Largest Withdrawal: 7000
```

Run Save

Your Code

```python
passwords = [
    "Python@123",
    "python123",
    "PYTHON@123",
    "Py@45",
    "Python123",
    "Java#4567",
    "Admin@2026"
]

valid = []
invalid = []

for password in passwords:
    conditions = []

    if len(password) < 8:
        conditions.append('Less than 8 characters')
    if not any(c.isupper() for c in password):
        conditions.append('No uppercase')
    if not any(c.islower() for c in password):
        conditions.append('No lowercase')
    if not any(c.isdigit() for c in password):
        conditions.append('No digit')
    if not any(not c.isalnum() for c in password):
        conditions.append('No special character')

    if len(conditions) == 0:
        valid.append(password)
    else:
        invalid.append((password, conditions))

print('Valid Passwords:')
print(valid)

print('\nInvalid Passwords and Failed Conditions:')
for password, conditions in invalid:
    print(password, '->', conditions)

print('\nNumber of Valid Passwords:', len(valid))
print('Number of Invalid Passwords:', len(invalid))

longest = max(passwords, key=len)
print('\nLongest Password:', longest)
print('Length:', len(longest))

python_passwords = [p for p in passwords if 'python' in p.lower()]
print('\nPasswords containing 'python':')
print(python_passwords)

sorted_passwords = sorted(passwords, key=len)
print('\nPasswords sorted by length:')
print(sorted_passwords)

percentage = (len(valid) / len(passwords)) * 100
print('\nPercentage of Valid Passwords:', percentage, '%')
```

Input

```
stdin input
```

Output

```
Valid Passwords:
['Python@123', 'Java#4567', 'Admin@2026']

Invalid Passwords and Failed Conditions:
python123 -> ['No uppercase', 'No special character']
PYTHON@123 -> ['No lowercase']
```

Run Save

CO4_AT2_CASE STUDY

← Back

QUESTIONS

Q1 Student Performance Analysis — List + Tuple 10 marks

Q2 Customer Name Analysis — String + List 10 marks

Q3 Product Inventory Analysis — List + Tuple 10 marks

Q4 Text Analysis — String + List 10 marks

Q5 Employee Skill Analysis — Tuple + List + String 10 marks

Q6 Transaction Analysis — Tuple + List 10 marks

Q7 Password Security Analysis — String + List 10 marks

Q8 Code Analysis and Debugging — List + Tuple + String 10 marks

Q9 Word Frequency and Data Analysis — String + List + Tuple 10 marks

Q10 Integrated Student Data Analysis — String + List + Tuple 10 marks

10/10 answered

Q8 Code Analysis and Debugging — List + Tuple + String

10 marks

Consider the following program:

```
records = [
(" Arun ", "Python"),
("ARUN", "Python"),
(" Bala ", "Java"),
("bala", "Java"),
("Charan", "Python")
]

names = []

for record in records:
name = record[0].strip().lower()

if name not in names:
names.append(name)

print(names)
```

Analyze the program:

1. What happens during each iteration?
2. What value is stored in record?
3. What does record[0] represent?
4. Why are strip() and lower() used together?
5. Why does "Arun" appear only once?
6. Why does "Bala" appear only once?
7. What is the final output?
8. Modify the logic to find unique employees and their skills.
9. Analyze what happens if record[0] is changed instead.
10. Explain why modifying record[0] directly is not allowed.

Expected Output:

['arun', 'bala', 'charan']

Your Code

```python
records = [
    (" Arun ", "Python"),
    ("ARUN", "Python"),
    (" Bala ", "Java"),
    ("bala", "Java"),
    ("Charan", "Python")
]

names = []

for record in records:
    name = record[0].strip().lower()
    if name not in names:
        names.append(name)

for name in names:
    print(name)
```

Input

```
stdin input
```

Output

```
arun
bala
charan
```

▶ Run Save

```
    else:
        names.append(x[1])

print("\n9. Duplicate names:")
print(duplicates)


sorted_students = sorted(data, key=lambda x: x[4])

print("\n10. Sorted by average:")
for x in sorted_students:
    print(x[1], x[4])


top_two = sorted(data, key=lambda x: x[4], reverse=True)[:2]

print("\n11. Top two students:")
for x in top_two:
    print(x[1], x[4])


highest_subject = []

for i in range(3):
    highest_subject.append(max(x[2][i] for x in data))

print("\n12. Subject-wise highest marks:")
print(highest_subject)


subject_average = []

for i in range(3):
    total = sum(x[2][i] for x in data)
    subject_average.append(total / len(data))

print("\n13. Subject-wise average marks:")
print(subject_average)


count = 0

for x in data:
    above_80 = 0

    for mark in x[2]:
        if mark > 80:
            above_80 += 1

    if above_80 >= 2:
        count += 1

print("\n14. Students above 80 in at least two subjects:")
print(count)


highest_subject3 = max(data, key=lambda x: x[2][2])

print("\n15. Highest Subject 3:")
print(highest_subject3[1], highest_subject3[2][2])


print("\n16. Data Structures:")
print("Student record: Tuple")
print("Marks: List")
print("Student identity: Student ID")
```

Input

```
stdin input
```

Output

```
1. Normalized IDs:
A001
A002
A003
A004
A005
```

Run Save